All Figures Preview

Unverified Assumptions in Embedded Safety Firmware:
Configuration, Specification and Response-Time Defects in an MQTT Alarm System

MD Humayun • SVNIT Surat • commit `1b0b62f`

Every figure is the exact file the manuscript embeds, shown at the width it occupies there (`\textwidth`). Figures 1–8 are the manuscript's; the figures after them document the repository and are not part of the submission.

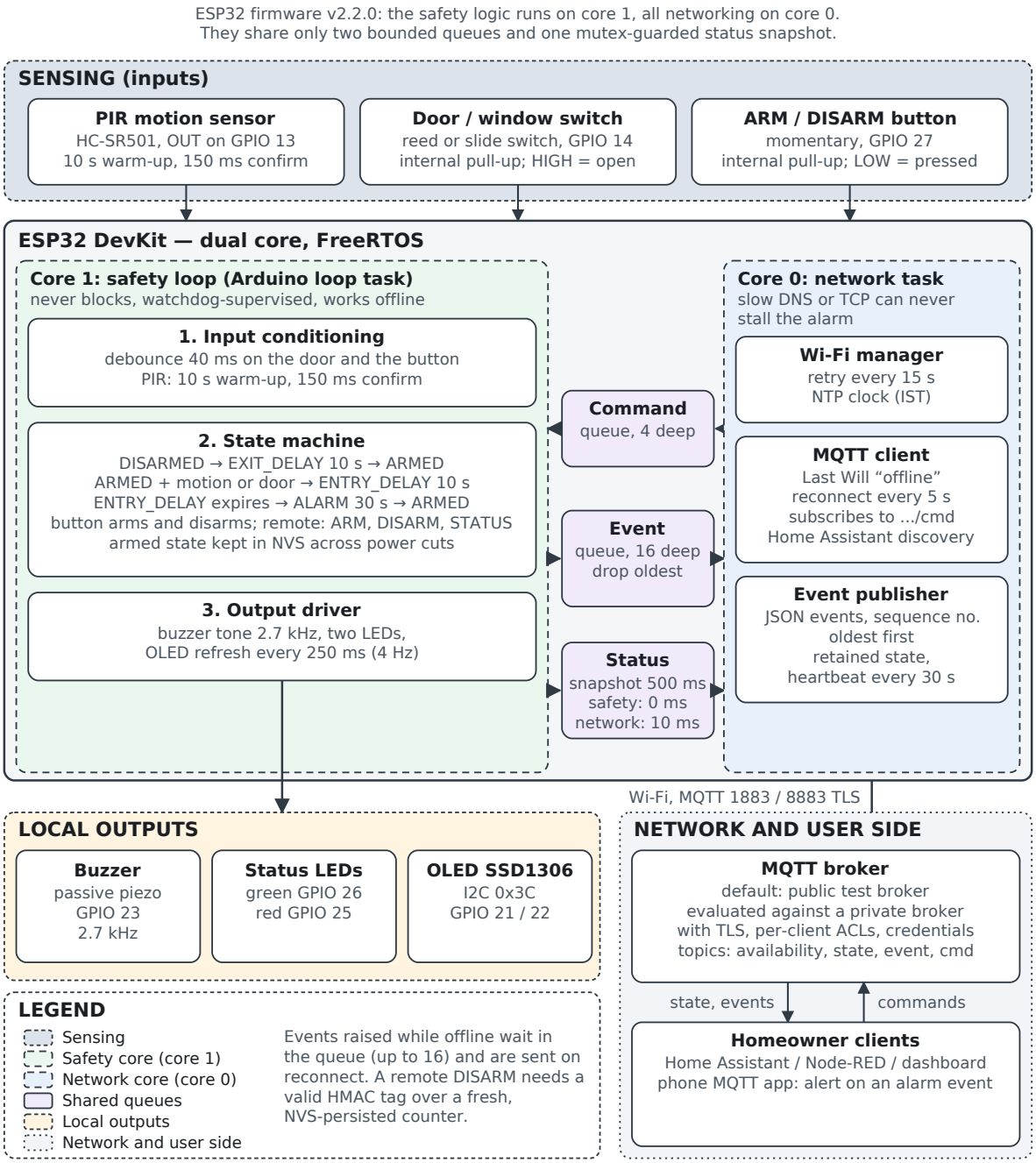
Contents

1 Manuscript figures **2**
Figure 1 — System architecture . . . 2
Figure 2 — Command execution flow . . . 3
Figure 3 — Command-path ablation results . . . 4
Figure 4 — Broker access control architecture . . . 5
Figure 5 — Security compromise comparison . . . 6
Figure 6 — Experimental setup . . . 7
Figure 7 — Compile-guard comparison . . . 8
Figure 8 — Transport security versus command authorization . . . 9

2 Additional repository figures **10**
Repository figure — Command sequence (UML) . . . 10
Repository figure — Deployment . . . 11
Repository figure — Methodology . . . 12
Repository figure — Alarm state machine . . . 13
Repository figure — Event sequence . . . 14
Repository figure — Circuit wiring . . . 15
Repository figure — Hardware measurement rig — NOT BUILT, NOT RUN . . . 16

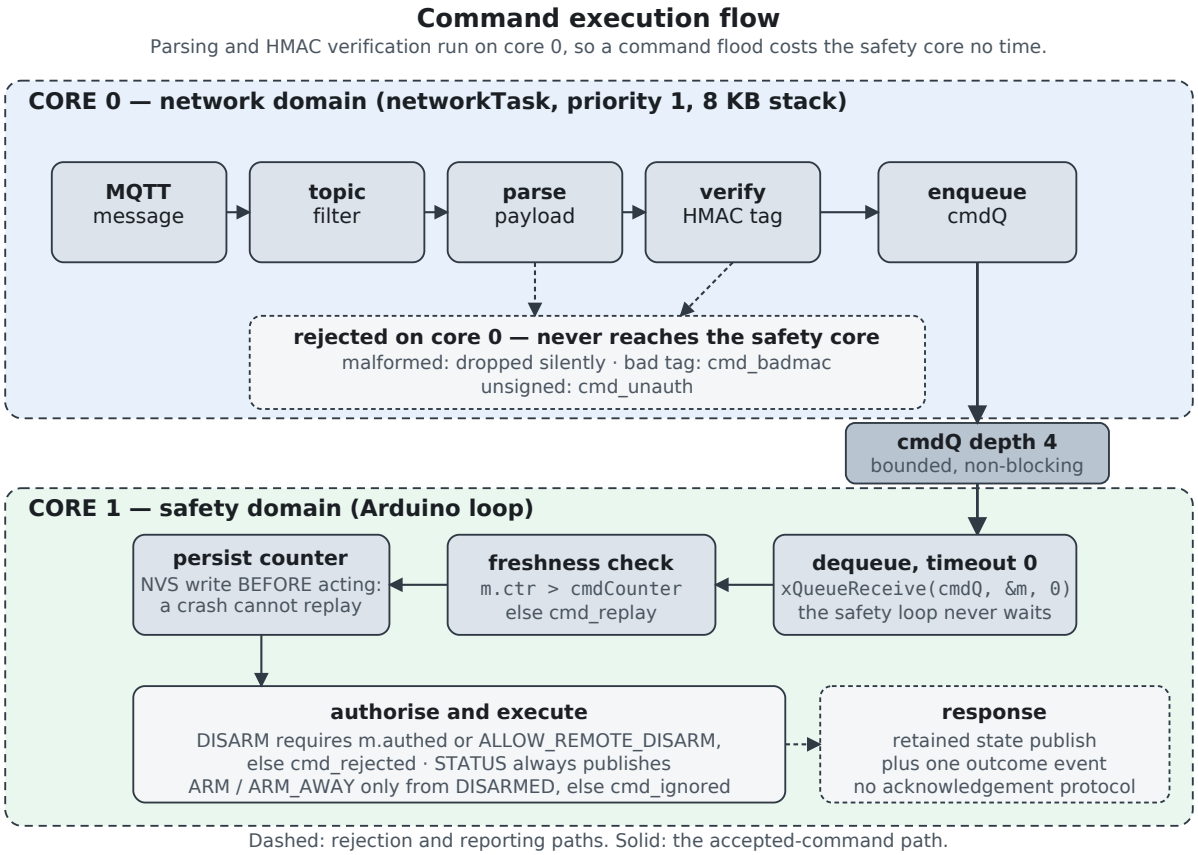
1 Manuscript figures

Figure 1 — System architecture



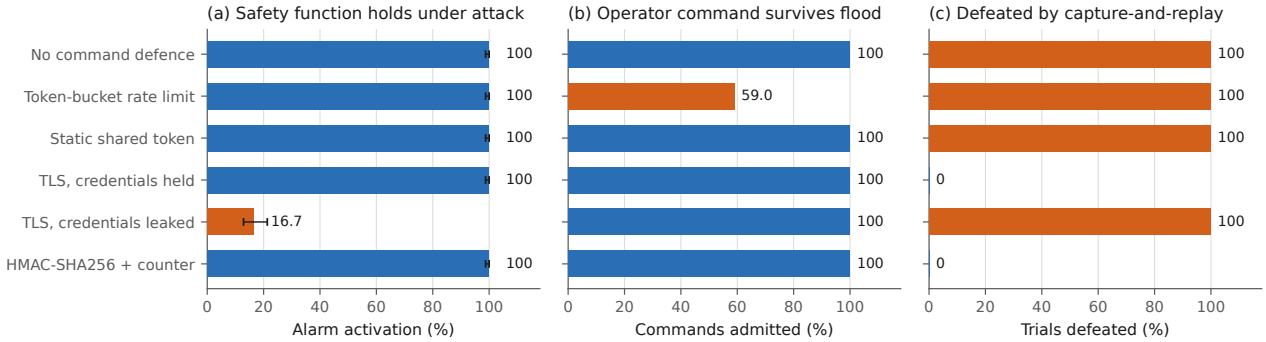
The safety domain on core 1 and the network domain on core 0, the two bounded queues and the mutex-guarded status snapshot between them. Every pin, timeout and queue depth is transcribed from `smart_home.ino`. *Manuscript page 8.*

Figure 2 — Command execution flow



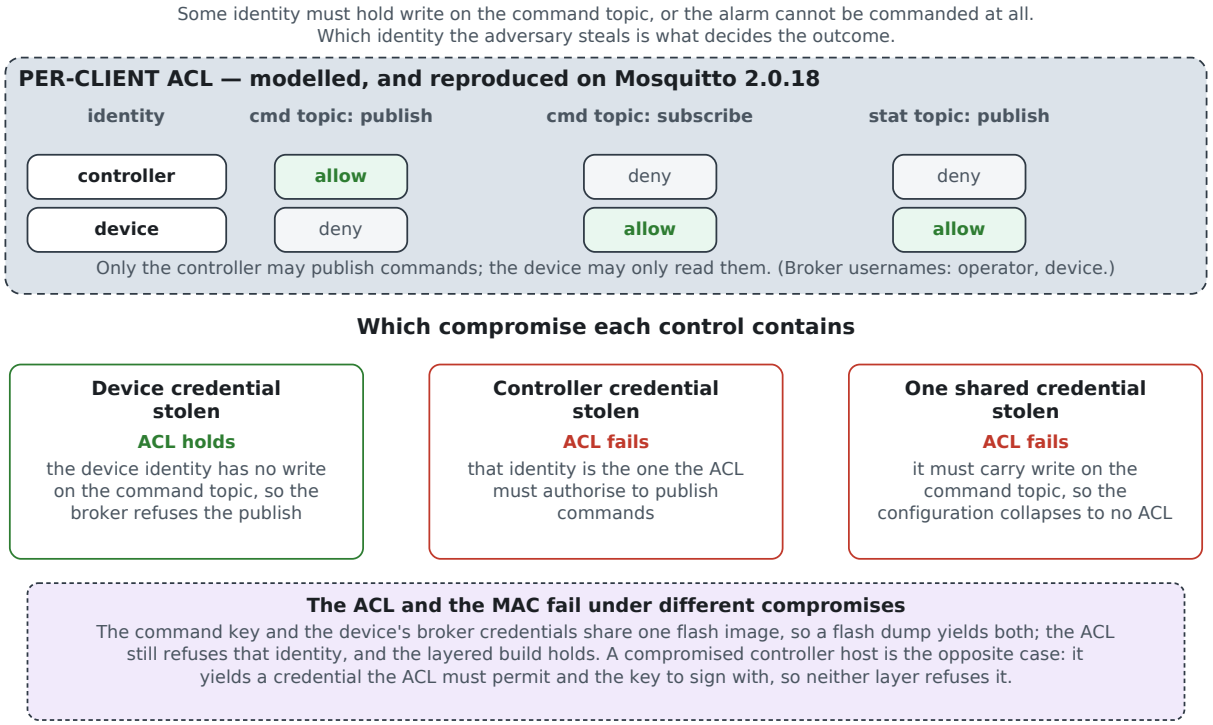
Parsing and HMAC verification complete on core 0; the replay counter is committed to NVS before the command is acted on. Rejected and accepted paths are distinguished. *Manuscript page 9.*

Figure 3 — Command-path ablation results



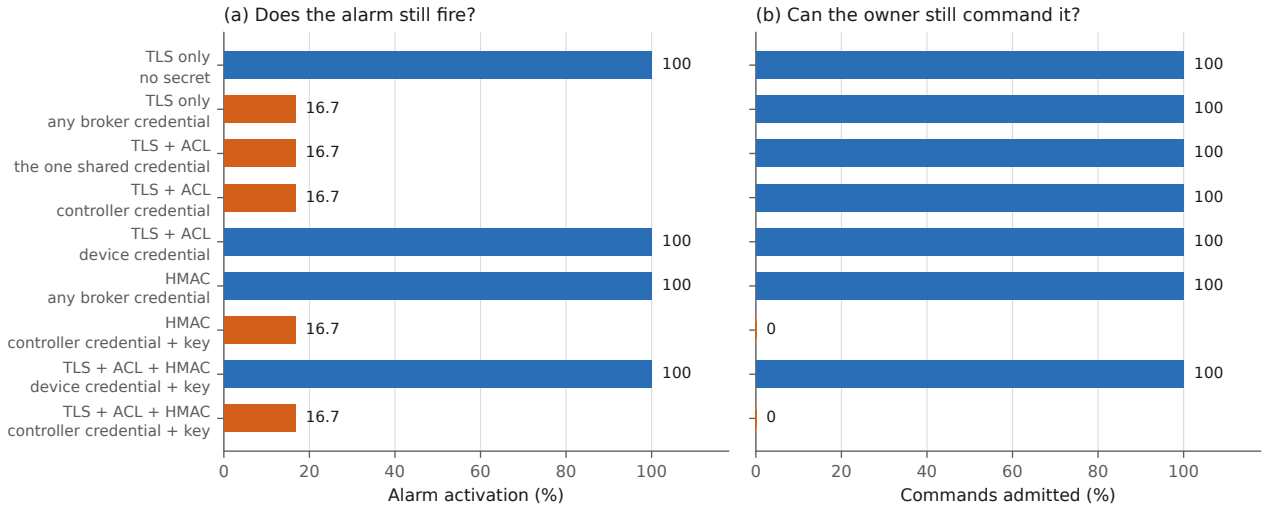
Six designs over three properties. 300 trials per bar in (a) and (c), 1,250 operator attempts per bar in (b); whiskers in (a) are 95 % Wilson intervals. *Manuscript page 16.*

Figure 4 — Broker access control architecture



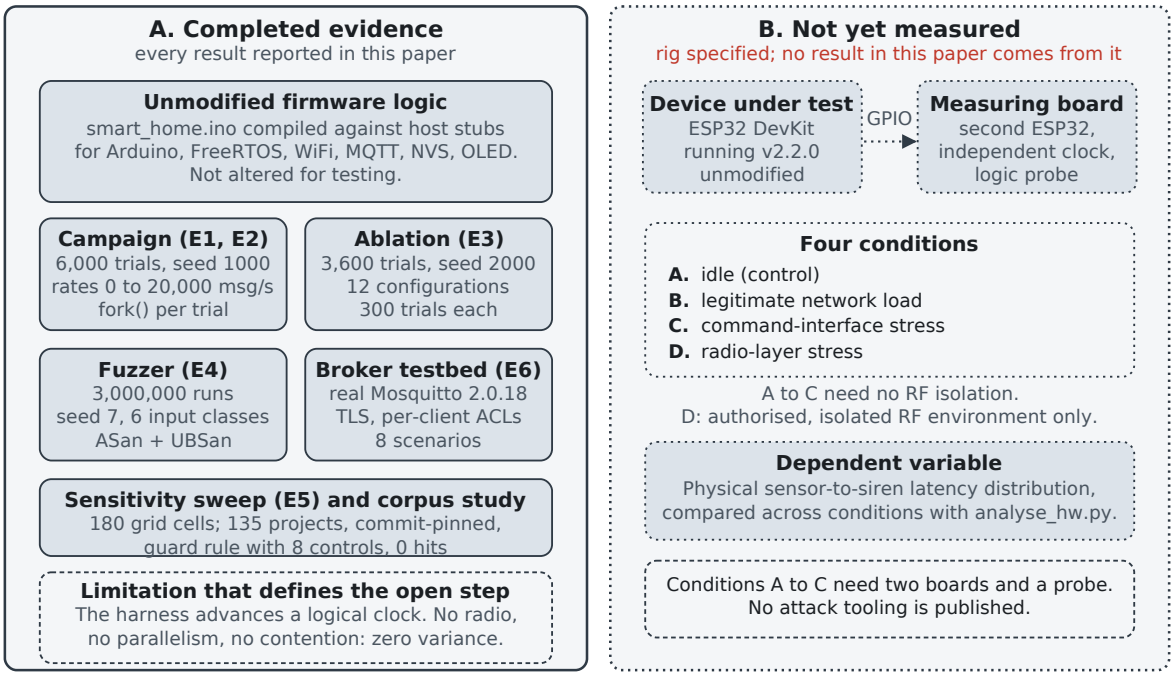
Per-client topic permissions, and which compromise each control contains. The matrix is both the one the ablation models and the Mosquitto `acl_file` used in the broker experiment. *Manuscript page 18.*

Figure 5 — Security compromise comparison



Alarm activation and operator availability for nine combinations of control and compromised secret. The bottom row is the layered build under a compromised controller host: it fails, and is drawn at the same weight as every other row. *Manuscript page 18.*

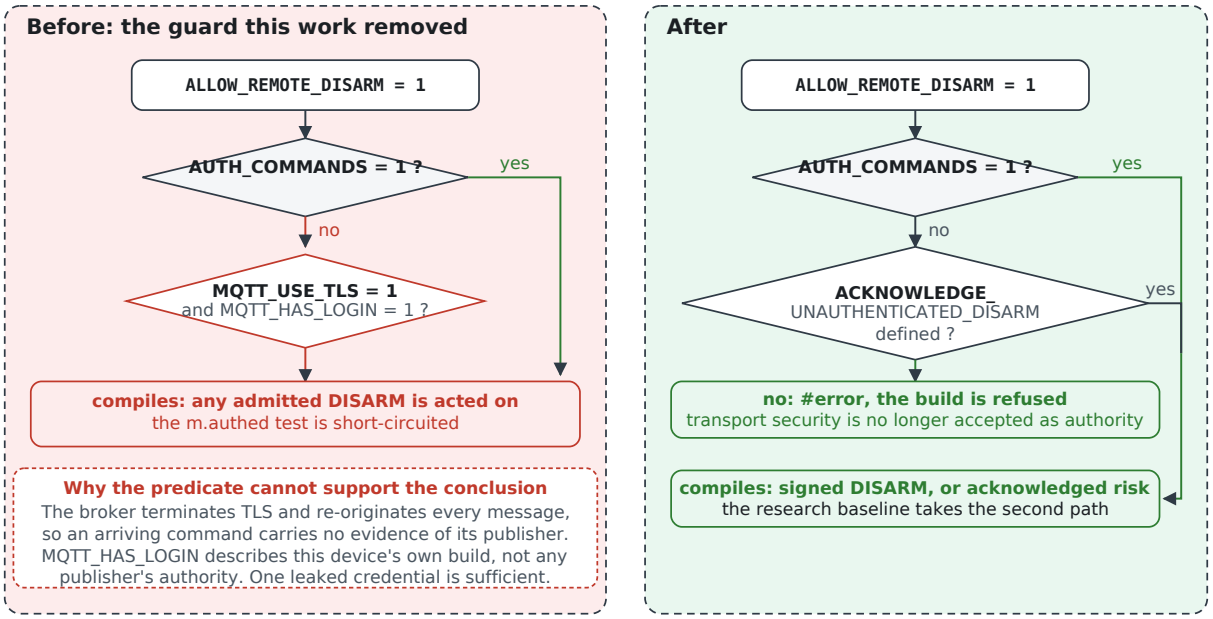
Figure 6 — Experimental setup



Panel A is everything that has been run. Panel B is the hardware rig: specified, not run, drawn dotted, and labelled in red that no result in the paper comes from it. *Manuscript page 22.*

Figure 7 — Compile-guard comparison

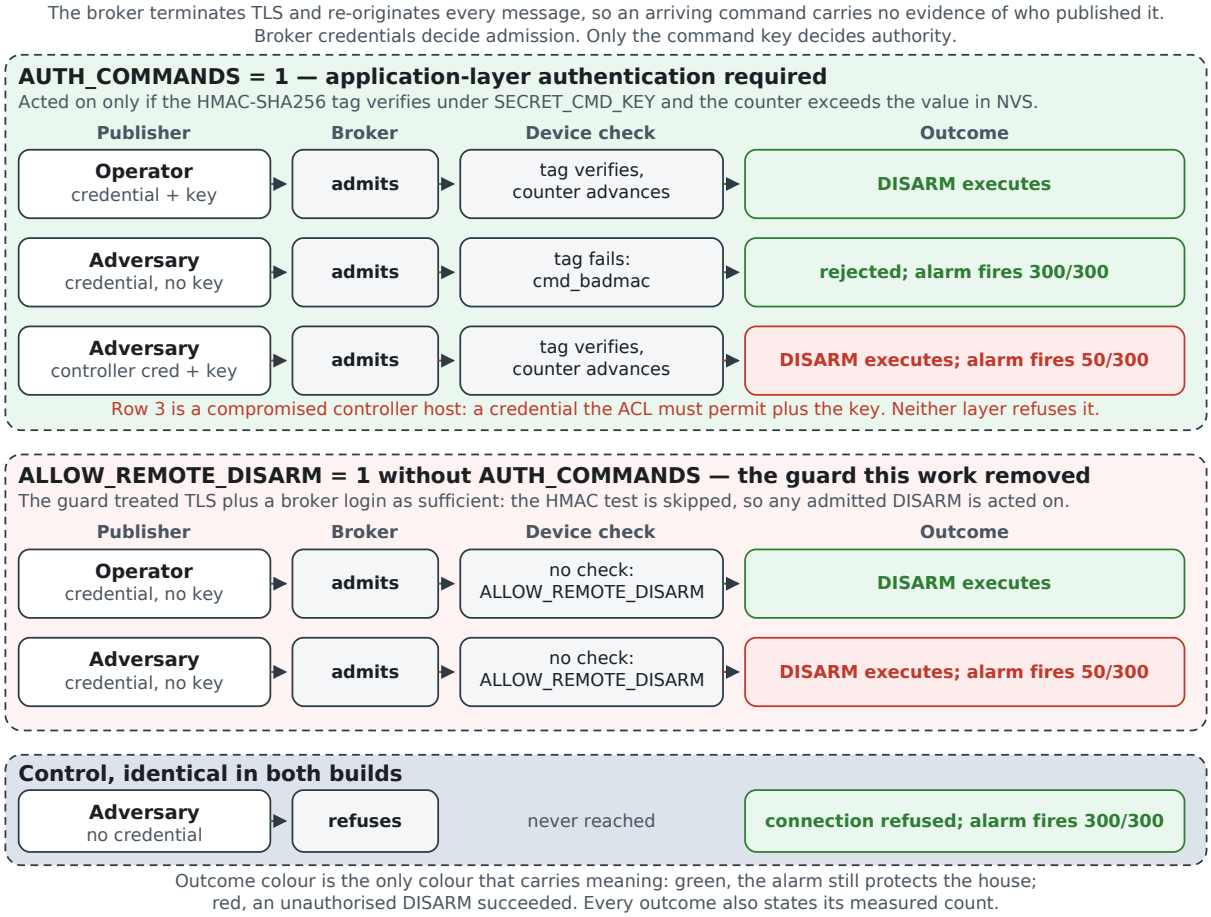
A build condition is a security argument. This one asserted that transport security plus a broker login implies publisher authority.



Red marks a build in which an unauthenticated DISARM can be acted on; green marks one in which it cannot.

The defective guard and its repair, transcribed from the preprocessor conditions. Both branches of the original reach the same outcome, which is the defect. *Manuscript page 24.*

Figure 8 — Transport security versus command authorization

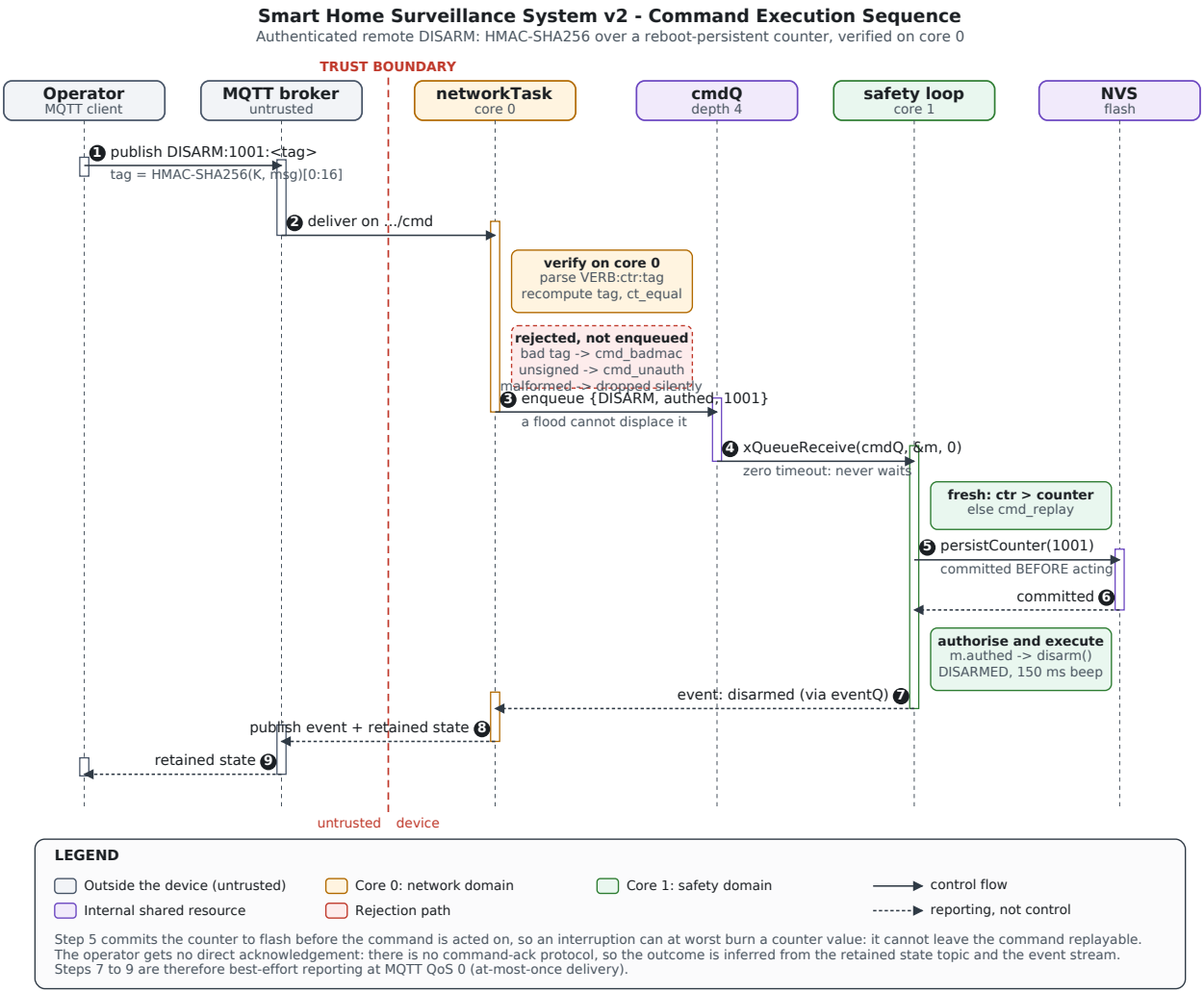


The third row of the upper panel is the compromised controller host, holding both a permitted credential and the key: neither layer refuses it, and the alarm fires in only 50 of 300 trials. *Manuscript page 25.*

2 Additional repository figures

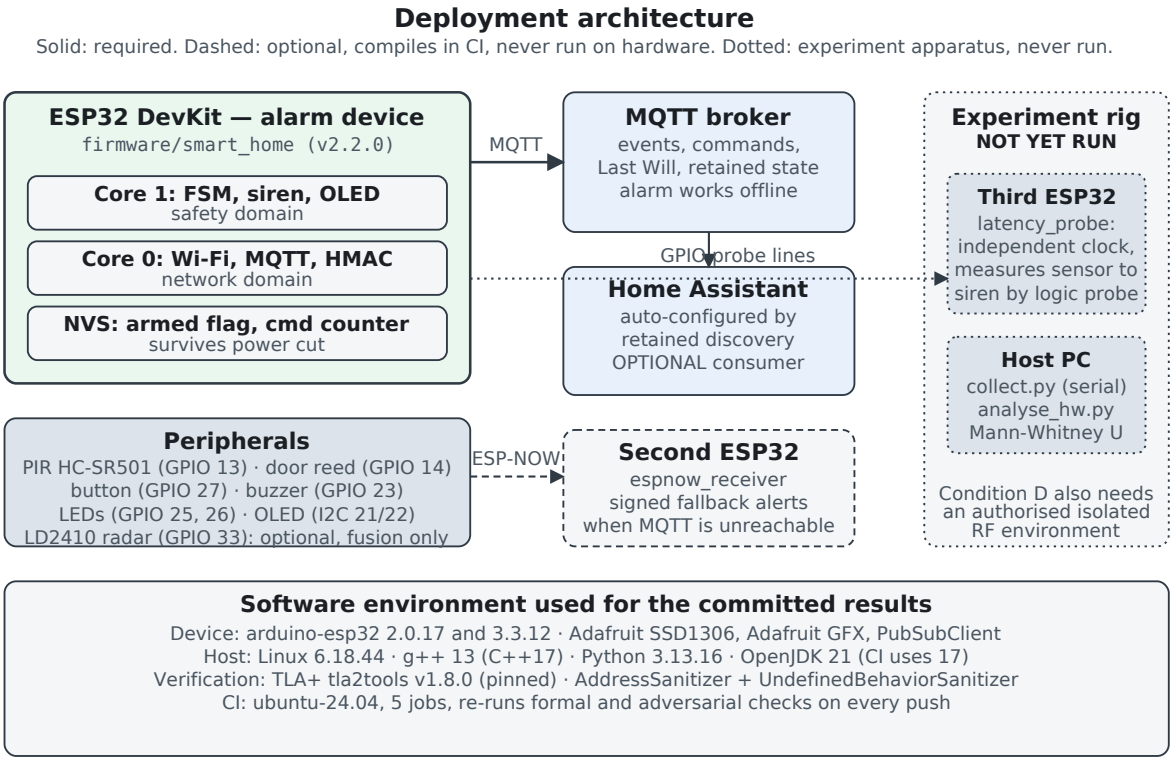
Not part of the manuscript.

Repository figure — Command sequence (UML)



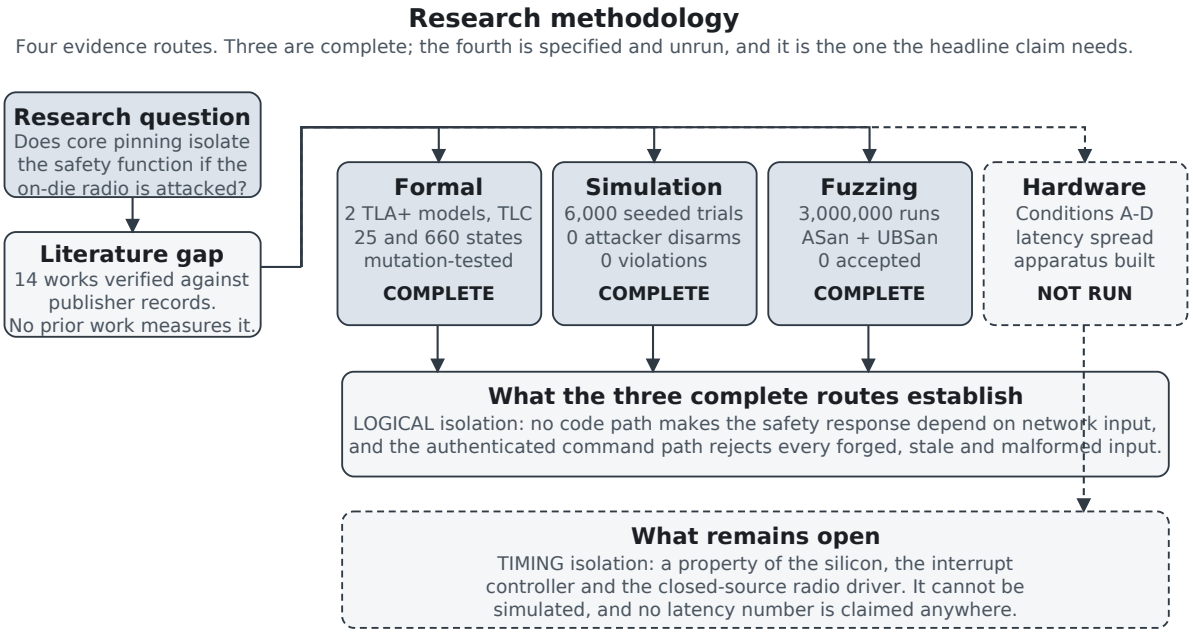
Documentation figure; not part of the manuscript.

Repository figure — Deployment



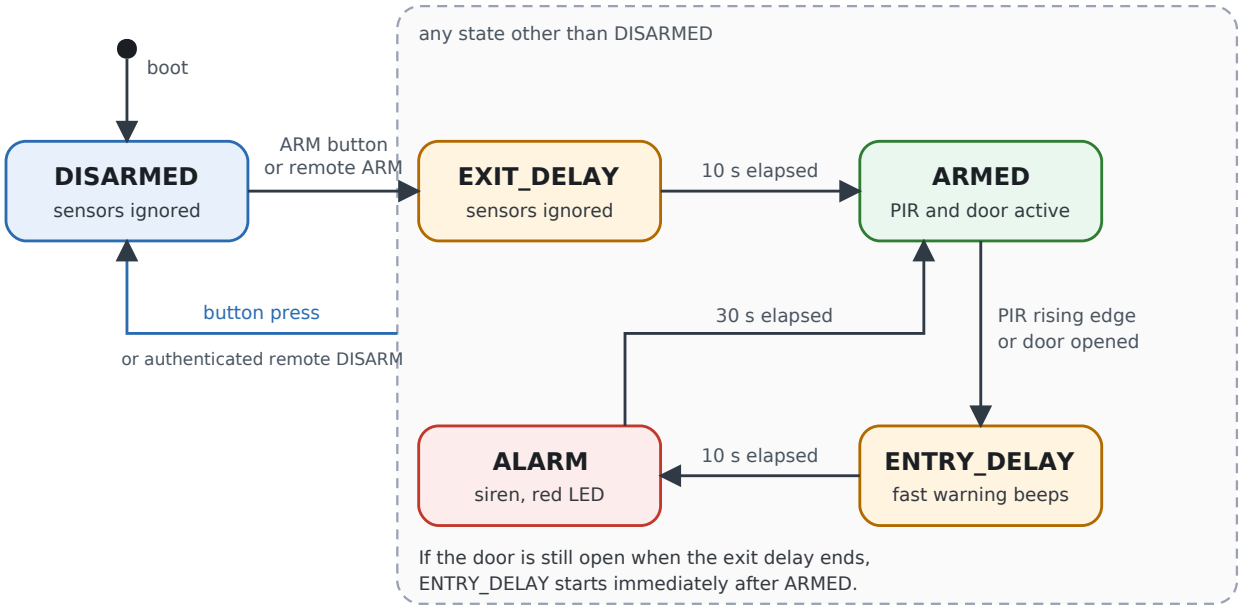
Documentation figure; not part of the manuscript.

Repository figure — Methodology



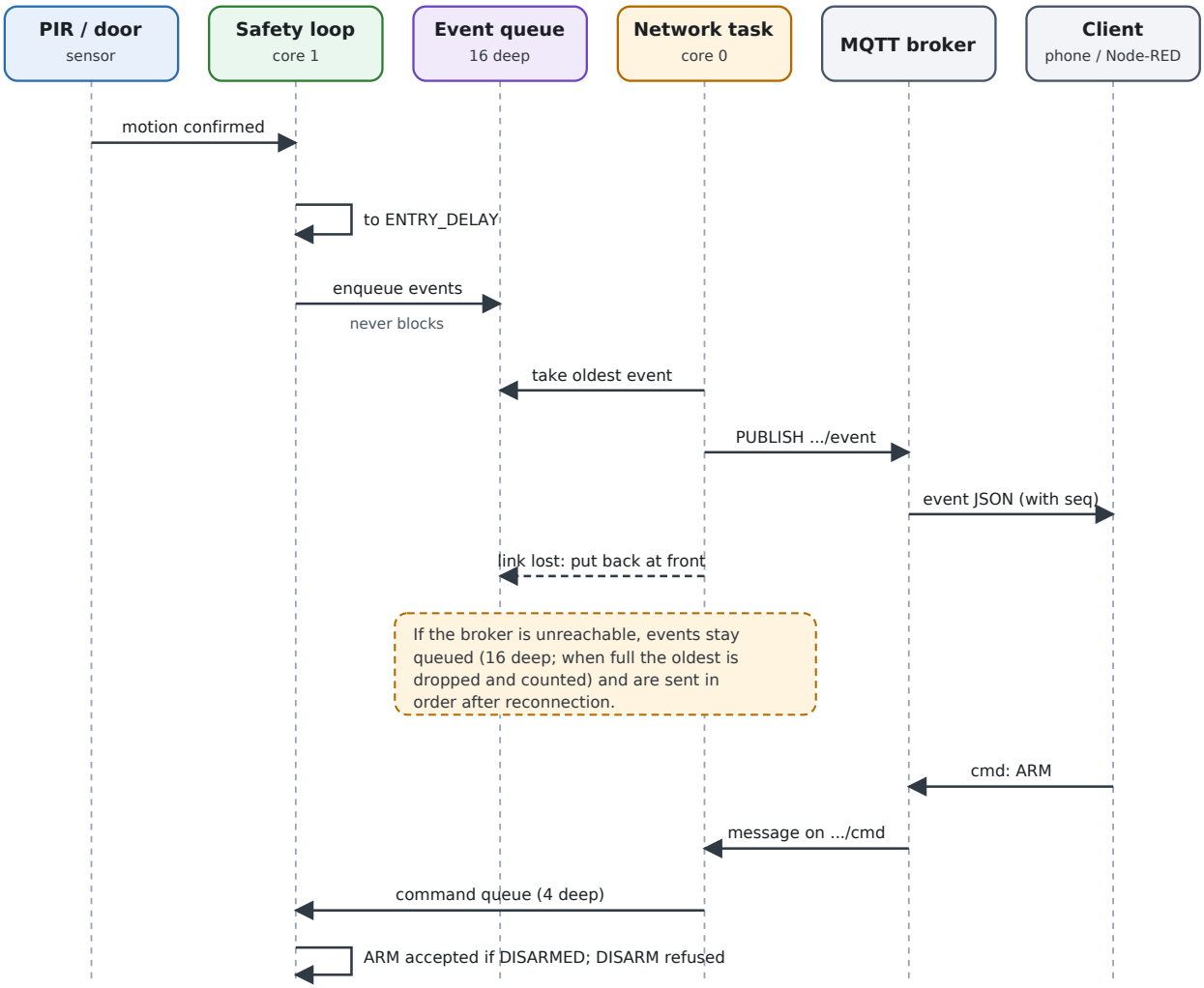
Documentation figure; not part of the manuscript.

Repository figure — Alarm state machine



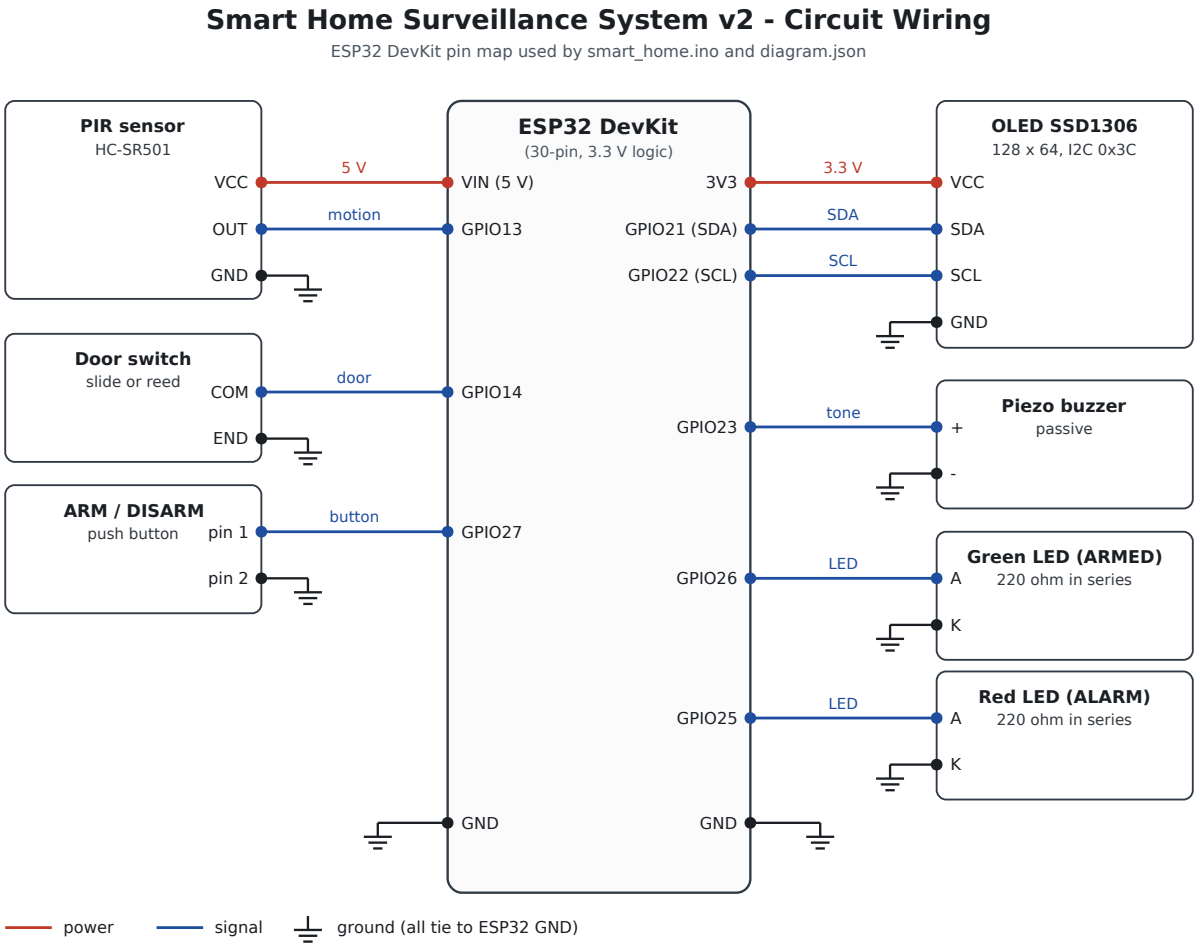
Documentation figure; not part of the manuscript.

Repository figure — Event sequence



Documentation figure; not part of the manuscript.

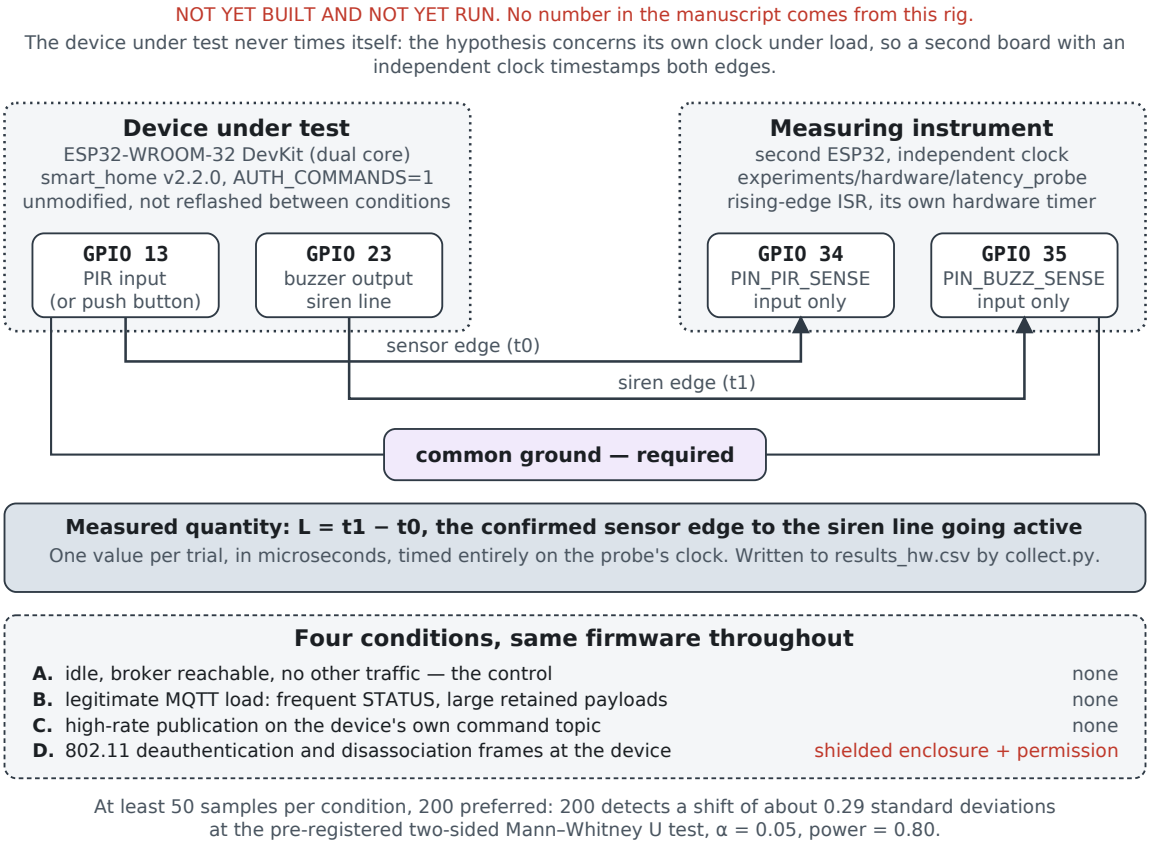
Repository figure — Circuit wiring



Door switch and button use INPUT_PULLUP: closed / pressed pulls the pin LOW.
The PIR needs 5 V (VIN); its OUT pin is 3.3 V logic, so it is safe for the ESP32.
Use a 220 ohm resistor in series with each LED. None of the pins used here is an ESP32 boot-strapping pin (GPIO0, 2, 5, 12 and 15 are avoided on purpose).

Documentation figure; not part of the manuscript.

Repository figure — Hardware measurement rig — NOT BUILT, NOT RUN



Documentation figure; not part of the manuscript.